

Experiment No 5:DataAnalytics II

1. Implement logistic regression using Python/R to perform classification on Social_Network_Ads.csv dataset.
2. Compute Confusion matrix of find TP,FP,TN,FN,Accuracy,Error rate,Precision, Recall onthe given dataset.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
dataset = pd.read_csv('/content/Social_Network_Ads.csv')
```

```
dataset.head()
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

Next steps: [Generate code with dataset](#) [New interactive sheet](#)

```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   User ID                400 non-null   int64
1   Gender                 400 non-null   object
2   Age                    400 non-null   int64
3   EstimatedSalary        400 non-null   int64
4   Purchased              400 non-null   int64
dtypes: int64(4), object(1)
memory usage: 15.8+ KB
```

```
dataset.isnull().sum()
```

	0
User ID	0
Gender	0
Age	0
EstimatedSalary	0
Purchased	0

dtype: int64

```
dataset.shape
```

```
(400, 5)
```

Description of below code

: → selects all rows

[2, 3] → selects column index 2 and column index 3

.values → converts the selected data into a NumPy array

```
x = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, 4].values
```

```
print(x)
```

```
[ 38 65000]
[ 47 51000]
[ 47 105000]
[ 41 63000]
[ 53 72000]
[ 54 108000]
[ 39 77000]
[ 38 61000]
[ 38 113000]
[ 37 75000]
[ 42 90000]
[ 37 57000]
[ 36 99000]
[ 60 34000]
[ 54 70000]
[ 41 72000]
[ 40 71000]
[ 42 54000]
[ 43 129000]
[ 53 34000]
[ 47 50000]
[ 42 79000]
[ 42 104000]
[ 59 29000]
[ 58 47000]
[ 46 88000]
[ 38 71000]
[ 54 26000]
[ 60 46000]
[ 60 83000]
[ 39 73000]
[ 59 130000]
[ 37 80000]
[ 46 32000]
[ 46 74000]
[ 42 53000]
[ 41 87000]
[ 58 23000]
[ 42 64000]
[ 48 33000]
[ 44 139000]
[ 49 28000]
[ 57 33000]
[ 56 60000]
[ 49 39000]
[ 39 71000]
[ 47 34000]
[ 48 35000]
[ 48 33000]
[ 47 23000]
[ 45 45000]
[ 60 42000]
[ 39 59000]
[ 46 41000]
[ 51 23000]
[ 50 20000]
[ 36 33000]
[ 49 36000]]
```

✓ This function is used to split the dataset into training and testing sets.

It helps evaluate how well a Machine Learning model performs on unseen data. $x \rightarrow$ independent variables / features $y \rightarrow$ dependent variable / target These two are split together, so each x sample correctly matches its y label.

`random_state = 0` Controls the randomness of the split Ensures reproducibility Using the same `random_state` always gives the same train-test split

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.30, random_state = 0)
```

✓ Feature Scaling?

Many ML algorithms (e.g., Logistic Regression, SVM, KNN) are sensitive to the scale of features. If one feature has large values and another has small values, the model may become biased. Standardization puts all features on the same scale.

`sc = StandardScaler()` Creates a `StandardScaler` object. This object will learn the mean and standard deviation from the data.

`x_train = sc.fit_transform(x_train)` `fit` → calculates mean & standard deviation from training data. `transform` → scales the training data using those values. Both steps combined using `fit_transform`. Scaling is learned only from training data to avoid data leakage.

This code standardizes features so that all variables have mean 0 and standard deviation 1, improving model performance and fairness.

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)
```

✦ classifier = LogisticRegression(random_state = 0)

Creates a Logistic Regression model object. `random_state = 0` ensures reproducible results. The model will learn how input features relate to output classes.

```
from sklearn.linear_model import LogisticRegression
classifier = LogisticRegression(random_state = 0)
classifier.fit(x_train, y_train)
```

▼ **LogisticRegression** ⓘ ?
LogisticRegression(random_state=0)

✦

`.predict()` uses the trained Logistic Regression model.

It predicts the class labels for the given input data.

`x_test` contains unseen (test) feature data. This data was not used during training. It helps check how well the model generalizes.

`y_pred` stores the predicted output values. These are the model's guesses based on learned patterns.

```
y_pred = classifier.predict(x_test)
```

✦ print(y_pred)

Displays the predicted class labels generated by the model. These are the outputs predicted from `x_test`.

```
print(y_pred)
```

```
[0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 1 0 0 0 0 0 0 1 0 0 0 0
 0 0 1 0 0 0 0 1 0 0 1 0 1 1 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0
 0 0 1 0 1 1 1 1 0 0 1 1 0 1 0 0 0 1 0 0 0 0 0 0 0 1 1 0 0 0 0 1 1 0 0 0 0 0
 0 0 1 1 1 1 0 1 1]
```

✦ print(y_test)

Displays the actual (true) class labels. These are the correct answers from the dataset.

```
print(y_test)
```

```
[0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 0 0 0
 0 0 1 0 0 0 0 1 0 0 1 0 1 1 0 0 0 1 1 0 0 1 0 0 1 0 0 1 0 1 0 0 0 0 1 0 0 1
 0 0 0 0 1 1 1 0 0 0 1 1 0 1 1 0 0 1 0 0 0 1 0 1 1 1 0 1 0 1 1 1 0 0 0 0 0
 0 1 1 1 0 1 0 0 1]
```

✦

`confusion_matrix(y_test, y_pred)` Takes: `y_test` → actual (true) labels, `y_pred` → predicted labels. Returns a 2x2 matrix for binary classification.

Where: TN (True Negative) → Correctly predicted 0 FP (False Positive) → Incorrectly predicted 1 FN (False Negative) → Incorrectly predicted 0 TP (True Positive) → Correctly predicted 1

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[74  5]
 [11 30]]
```

```
Accuracy=(74+30)/120
Accuracy
```

```
0.8666666666666667
```

```
Error_rate=(5+11)/120
Error_rate
```

```
0.13333333333333333
```

▼ This means:

👉 86% predictions are correct

```
from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_pred)
```

```
0.8666666666666667
```

▼ Precision measures how many of the positive predictions made by the model are actually correct.

Formula:

Precision= TP+FP/TP

Where: TP (True Positive) → Correctly predicted positive cases FP (False Positive) → Incorrectly predicted positive cases 85.71 % - Correctness of positive predictions (positive predictions made by the model are actually correct.)

```
from sklearn.metrics import precision_score, recall_score
precision_score(y_test, y_pred)
```

```
0.8571428571428571
```

▼ Recall measures how many of the actual positive cases were correctly identified by the model.

Formula:

Recall=TP+FN/TP

Where: TP (True Positive) → Correctly predicted positives FN (False Negative) → Positives predicted as negative FN (False Negative) → Positives predicted as negative

```
recall_score(y_test, y_pred)
```

```
0.7317073170731707
```

▼ f1_score(y_test, y_pred) do?

Compares: $y_{\text{test}} \rightarrow$ actual labels $y_{\text{pred}} \rightarrow$ predicted labels Computes the harmonic mean of precision and recall By default, calculates F1-score for positive class (1)

```
from sklearn.metrics import f1_score  
f1_score(y_test, y_pred)
```

```
0.7894736842105263
```

Summary Table

Metric What it measures Precision Correctness of positive predictions Recall Coverage of actual positives F1-score Balance of precision & recall

```
from google.colab import files  
  
uploaded = files.upload()  
  
if uploaded:  
    print("Success")  
else:  
    print("No file uploaded")
```

No file chosen
No file uploaded